ENTERPRISE ARCHITECTURE · WHITE PAPER

Mastering AI in Production

Eighteen Design Patterns for Industry-Grade AI in Banking, Financial Services & Regulated Enterprises

Every durable system begins not with code but with a story — a narrative of forces in tension: the need to move fast against the imperative to be safe; the promise of intelligence against the weight of regulation; the allure of the new against the governance of the established. This paper tells that story through the vocabulary of design patterns: the hard-won grammar of architects who have built AI systems that survive contact with production, with auditors, and with real money on the line.

L1	L2	L3	L4
Foundation	**Orchestration**	**Integration**	**Governance**
Reasoning primitives inside a single agent	How multiple agents coordinate complex work	Connecting AI to the enterprise data fabric	Safe, auditable, resilient at scale

Banking & Financial Services · 2026

Aligned to APRA CPS 230 / CPS 234 · ASIC RG 271 · MAS TRM · Privacy Act 1988 · ISO/IEC 42001 · EU AI Act

Contents

Executive Summary

Artificial intelligence has crossed the threshold from experiment to infrastructure. In banking and financial services, language models and agentic systems now sit close to processes that move money, decide credit, screen for financial crime, and answer regulated customer questions. That proximity changes everything. A demonstration that impresses a steering committee is not the same artefact as a system that an auditor, a prudential regulator, and an on-call architect can all stand behind at three in the morning. **The distance between those two artefacts is architecture — and the language of that architecture is design patterns.**

This white paper presents eighteen patterns for building AI systems that endure in regulated production. The patterns are organised into four layers that build on one another. The Foundation layer governs how a single agent reasons, retrieves, acts, and remembers. The Orchestration layer defines how multiple specialised agents coordinate without corrupting each other's work. The Integration layer connects intelligence to the existing enterprise fabric — the core banking platform, the event bus, the API gateway, the human workforce. The Governance layer is the operating wrapper that makes the whole system safe, observable, explainable, and economically sustainable.

The central argument is simple and uncomfortable: **most AI production failures are not model failures. They are architecture failures.** An agent loops forever because nobody defined a stopping condition. A decision cannot be explained to a regulator because the reasoning was stripped from the log. A semantic cache returns one customer's balance to another because nobody classified the query. A circuit breaker opens onto a generic error page rather than a meaningful fallback. None of these are exotic model behaviours. They are the predictable consequences of skipping patterns that the industry has already learned, often painfully.

Who this paper is for

This is a practitioner's reference written for enterprise architects, solution architects, engineering leads, risk and compliance officers, and the delivery governance functions that sit between board-level intent and production reality. It assumes familiarity with enterprise IT — service architecture, event-driven systems, API governance, change management — and translates that fluency into the specifics of agentic AI. Each pattern is presented with its mechanism, its concrete banking application, the most common mistake that has undone real projects, and the fix that prevents it.

How to read it

Read the layers in order. The sequencing is not arbitrary. **Foundation before Orchestration; Integration through the existing fabric rather than around it; Governance as a design primitive rather than a retrofit.** An organisation that masters Layer 1 and defers Layer 4 will build agents that reason beautifully and cannot be audited. An organisation that invests in orchestration before fixing its foundations will scale its instability. The patterns compose, but only in the right order.

Throughout, the regulatory frame is Australian and Asian financial services — APRA CPS 230 (operational risk management) and CPS 234 (information security), ASIC RG 271 (internal dispute resolution), the Monetary Authority of Singapore's Technology Risk Management guidelines, and the Privacy Act 1988 — with reference to the emerging baseline of ISO/IEC 42001 and the EU AI Act. The patterns themselves are jurisdiction-neutral; the obligations they help discharge are not.

> **The one-sentence thesis**
>
> *Treat an AI system the way you treat any other critical IT system: govern its changes, trace its decisions,*

> *bound its behaviour, and design its failure modes before they happen — because the regulator will not accept "the model did it" as an answer.*

Prologue: The Architect's Dilemma

It is three in the morning in a bank's war room in Sydney. A payment has failed — not crashed, not timed out, but made the wrong decision. The AI model trusted with real-time fraud scoring on the New Payments Platform rails declined a legitimate payment from a loyal customer. The customer called. Then complained. Then posted publicly. The incident ticket is already open, and the clock on the regulatory reporting window has already started.

The architect on call stares at the logs. There is an output — DECLINED — but the reasoning is gone, stripped in a pipeline optimisation three weeks earlier. The prompt version that ran? Unknown. Which retrieval chunks informed the decision? Not stored. Was the guardrail even active at that hour? Nobody is sure. The model can be queried again, but it is a different model now; the weights were refreshed last week. The single decision that mattered cannot be reconstructed.

This is not a story about AI failing. It is a story about the absence of patterns.

Design patterns in AI are not academic constructs. They are scar tissue — lessons encoded so they need not be relearned at three in the morning. They answer the questions that actually matter in production. How does an agent know when to stop? How do multiple agents divide work without corrupting each other's results? How does a regulated institution prove that a machine-made decision was correct, explainable, and compliant — not in principle, but for one specific customer, on one specific night, under one specific version of the system?

Consider what the architect in our story actually needed, and did not have. She needed a versioned prompt, so she could know exactly what instruction the model received. She needed a complete inference trace, so she could see the retrieved context and the reasoning, not merely the verdict. She needed a circuit breaker that routed to a meaningful fallback rather than an error. She needed a human-in-the-loop gate on high-value declines, so a person could have caught the error before the customer did. Every one of those is a pattern in this paper. Every one was missing.

The remainder of this document is organised as a journey through four layers of patterns. Each layer builds on the one beneath it. Together they form the architecture of an AI system that a bank, a regulator, and an architect can all trust — not merely in a demonstration, but at midnight on a Monday, under load, with real money on the line. The next architect on call at three in the morning will thank you for building it.

> **Why patterns, and why now**
>
> AI capability is advancing faster than AI governance maturity. The institutions that succeed will not be those with the most powerful models, but those with the most disciplined architecture around ordinary ones. Patterns are how that discipline is captured, taught, and enforced — the difference between an organisation that learns each lesson once and one that relearns it on every project.

The Four Layers: A Reference Model

Before descending into individual patterns, it helps to hold the whole structure in view. The four layers form a stack. Each answers a different architectural question, and each depends on the integrity of the layer beneath it.

Layer 1 — Foundation: can a single agent think?

The Foundation layer governs the reasoning primitives inside one agent: how it reasons step by step, how it grounds its answers in retrieved knowledge, how it acts through tools, how it checks its own work, how it remembers, and how its instructions are engineered. These are the atoms from which everything higher is built. An organisation that gets the foundation wrong does not get a slightly worse system; it gets a system whose every higher-order behaviour inherits the flaw.

Layer 2 — Orchestration: can many agents cooperate?

A single well-designed agent is a reliable worker. A network of poorly coordinated agents is an organisational disaster dressed up as innovation. The Orchestration layer defines the structure of multi-agent work: who plans, who executes, who reviews, and who decides that enough has been done. These patterns are the difference between a collection of agents and a system with genuine, governable capability.

Layer 3 — Integration: can AI touch the real world safely?

An AI agent with no connection to real data is a philosophical exercise. The Integration layer is where AI meets the core banking system, the event bus, the API gateway, and the human workforce. It is also where most enterprise AI projects either deliver genuine value or accumulate technical debt that outlasts the original team. The governing principle is restraint: AI should consume the enterprise fabric through its existing, already-governed patterns, not bypass them.

Layer 4 — Governance: can the institution stand behind it?

Governance is not the layer you add at the end. It is the operating wrapper that transforms a prototype into a system a chief information security officer, a risk committee, and a prudential regulator can endorse. It addresses safety, resilience, observability, and cost — the four ways a production AI system most commonly betrays the institution that deployed it.

> **The composition rule**
>
> **Patterns compose, but only upward and only in order.** You may combine a ReAct agent (L1) inside a Supervisor–Worker topology (L2), expose it through Tool Calling (L3), and wrap it in Guardrails and Observability (L4). You may not meaningfully add orchestration to an agent with no stopping condition, or governance to a system with no trace. Build the stack from the bottom.

The eighteen patterns at a glance

Pattern	L	Primary BFSI use case	The mistake to avoid
ReAct	L1	KYC enrichment, fraud investigation	No loop termination condition
Chain of Thought	L1	Credit decisioning, risk explainability	Stripping reasoning from the audit log

Pattern	L	Primary BFSI use case	The mistake to avoid
RAG	L1	Policy Q&A, regulatory lookup	Hard-split chunking breaks clauses
Self-Reflection	L1	Document generation, compliance review	Critiquing in the same context window
Memory Management	L1	Customer context, long-running cases	Storing raw PII in AI memory
Structured Prompting	L1	Every agent — the baseline	Unversioned prompts in application code
Planner–Executor	L2	Onboarding, loan origination	No DAG; tasks run out of order
Supervisor–Worker	L2	AI banking platform, multi-domain	Routing with no confidence fallback
Critic–Refiner	L2	Regulatory reports, communications	No iteration cap; infinite refinement
Map–Reduce	L2	Large contract & portfolio analysis	No chunk overlap; boundary failures
Tool / Function Calling	L3	Core banking API integration	Agent holds write access it does not need
Event-Driven AI	L3	NPP fraud scoring, AML monitoring	LLM latency exceeds the payment SLA
Semantic Cache	L3	FAQ deflection, rate lookups	Caching personalised responses
Human-in-the-Loop	L3	High-value & regulated decisions	HITL retrofitted, not a blocking gate
Guardrails Framework	L4	Every agent — the safety baseline	Guardrails only on external agents
Circuit Breaker	L4	Critical service resilience	Circuit opens to a generic error
AI Observability	L4	Regulatory audit trail	Logging output but not prompt context
Cost & Token Governance	L4	FinOps, model-tier routing	Top-tier model for every task

Layer 1 — Foundation Patterns

The reasoning primitives — what happens inside a single agent.

Before an AI system can coordinate across teams or integrate with enterprise APIs, it must first be able to think. The Foundation layer governs how a single agent reasons, retrieves, acts, and remembers. These are the atoms from which all higher-order patterns are built. Get them wrong and no amount of orchestration sophistication will save the system. Get them right and you have a trustworthy, auditable reasoning unit that can be composed into something genuinely powerful.

There is a temptation, common in enterprises under pressure to show progress, to skip straight to the visible, impressive work of multi-agent orchestration. This is a mistake that compounds. An agent that hallucinates because it has no grounding will hallucinate inside every workflow you place it in. An agent that loops because it has no stopping condition will loop more expensively when you give it more tools. The foundation is not the boring part to get past; it is the part that determines whether everything above it is sound.

> **Enterprise principle**
>
> **Get Layer 1 right before scaling to Layer 2 and beyond.** Most production failures trace back to a poorly designed foundation — an agent that hallucinates because it has no grounding, or loops because it has no stopping condition. Fix the primitive before adding orchestration complexity.

[ReAct] ReAct — Reason + Act

ReAct is the foundational loop of agentic AI. The agent alternates between reasoning — what do I know, what do I still need? — and acting — calling a tool or an API. Each observation feeds back into the next reasoning step, creating a closed decision loop that continues until a terminal condition is met. Where a traditional rule engine encodes every branch in advance, a ReAct agent decides its next move based on what it has just learned. That adaptivity is its power and, ungoverned, its danger.

How it works

The model produces a structured thought, then emits a tool call, receives an observation from the runtime, and continues. The cycle — Thought, Action, Observation — repeats until the agent reaches a defined stopping condition or produces a final answer. In banking this pattern replaces brittle, hand-maintained rule chains with adaptive, traceable decision sequences. The trace itself becomes valuable: each Thought is a contemporaneous record of why the agent did what it did, which is precisely what an auditor will later ask for.

```
// KYC Enrichment Agent — ReAct loop
Thought:  Customer ID is new. I need sanctions + PEP status.
Action:   sanctions_check({ entity_id: 'C-4821' })
Observation: CLEAR — no matches in OFAC / UN consolidated lists

Thought:  Now check PEP registry and adverse media.
Action:   pep_lookup({ name: 'John Smith', dob: '1975-03-14' })
Observation: PEP match — former regional councillor

Thought:  PEP match requires enhanced due diligence flag.
Final:    Risk tier = HIGH. Route to EDD queue.   [terminal action]
```

Why it matters in BFSI

KYC enrichment, fraud investigation, credit triage, and trade surveillance all share a shape: gather evidence from several systems, reason over it, and decide. ReAct fits this shape natively. Critically, because each step is explicit, the pattern produces an explainability artefact almost for free — provided the trace is preserved. The reasoning is not a by-product to be discarded; under APRA CPS 230 it is part of how you demonstrate that an automated operational process behaves as intended.

Mistake — no loop termination condition

The agent keeps calling tools after it already has enough information, burning tokens and latency, sometimes circling indefinitely. A forty-step loop on a simple KYC check is not a quirk; it is a production incident that surfaces in your CPS 230 operational-risk log and your inference bill at the same time.

Fix

Always define explicit stopping criteria. Set a hard step ceiling (for example max_steps = 8) and a terminal action (such as submit_risk_verdict) that halts the loop cleanly. Encode this in the system prompt and — crucially — enforce it in the runtime wrapper, not merely as an instruction the model is free to ignore. A stopping condition that lives only in the prompt is a suggestion; one that lives in the orchestration code is a guarantee.

BFSI use cases: *KYC enrichment · fraud investigation · credit decisioning · trade surveillance*

[CoT] Chain of Thought

Chain of Thought forces the model to externalise each reasoning step before producing a final answer. It improves accuracy on multi-step problems by giving the model room to work, and — the point that matters most in a regulated institution — it creates an audit trail. In banking, the chain of reasoning is not a quality tool that happens to be legible. It is the explainability artefact itself.

How it works

Zero-shot Chain of Thought adds a simple instruction such as “think step by step” to the prompt. Few-shot Chain of Thought supplies two or three worked examples that demonstrate the desired reasoning structure. For regulated decisions the structure should be prescribed rather than left to the model: tell it which factors to consider, in which order, so that every decision of a given type is reasoned the same way and can be compared across cases.

```
// System instruction for credit decisioning:
// "Reason through, in order: (1) income stability,
//  (2) debt-to-income ratio, (3) repayment history,
//  (4) collateral. Show each step explicitly."

Step 1 — Income:     $110k salary, 3 yrs same employer  -> STABLE
Step 2 — DTI:        $1,800/mo debt / $9,167 gross = 19.6%  -> PASS
Step 3 — History:    2 late payments in 48 months  -> MINOR FLAG
Step 4 — Collateral: Property $620k, LVR 68%  -> ADEQUATE
Decision: APPROVE at standard rate.  Flag: monitor repayment.
```

Why it matters in BFSI

When ASIC or APRA asks why a specific application was declined, “the model decided” is not an answer an institution can give. A persisted chain of reasoning lets the institution reconstruct the decision factor by factor, demonstrate that prohibited attributes played no role, and show consistency across similar cases. The same artefact supports internal model-risk review and customer remediation. Reasoning that is generated and then thrown away is reasoning the institution cannot account for.

Mistake — stripping the reasoning from the audit log

A developer trims the chain of thought from the response before logging, keeping only the final decision to save storage or tidy the output. Under APRA CPS 230 or MAS TRM, the inability to explain an automated decision is a compliance gap, not an implementation detail. The reasoning was the evidence, and it has been deleted.

Fix

Persist the full chain of reasoning to your decision-log store as structured output (`for example { reasoning: […steps], decision: …, confidence: 0.87 }`). Treat the reasoning trace with the same retention and integrity controls as any other record supporting a regulated decision. It is not a debug artefact; it is the audit trail.

BFSI use cases: *credit decisioning · risk explainability · insurance underwriting · regulatory Q&A*

[RAG] Retrieval-Augmented Generation

Retrieval-Augmented Generation decouples knowledge from model weights. Rather than relying on what the model absorbed during training — which is frozen, opaque, and quickly stale — RAG retrieves external documents at query time and injects them into the model's context. Policies, regulations, product specifications, and procedure manuals become live inputs. The pattern addresses hallucination and staleness simultaneously, which is why it is the single most important foundation pattern for knowledge-intensive banking work.

How it works

A RAG pipeline has six stages. Ingestion converts source documents into a vector store. At query time the user's question is embedded into the same vector space. Retrieval finds the most similar chunks. Augmentation assembles the prompt from system instructions, retrieved context, and the user query. Generation produces an answer grounded in that context. Finally, validation checks faithfulness — confirming the answer reflects the retrieved text rather than the model's memory.

```
// RAG pipeline for bank policy Q&A
1. Ingestion:    PDF -> semantic chunker -> embedder -> vector DB
2. Query time:   user_query -> embedder -> similarity search
3. Retrieval:    top-k chunks (k = 5, threshold = 0.72 cosine)
4. Augmentation: system_prompt + retrieved_chunks + user_query
5. Generation:   LLM -> answer WITH citations to source chunks
6. Validation:   faithfulness check of answer vs retrieved context
```

Why it matters in BFSI

Regulatory text changes; product terms change; the model's training does not. RAG lets an institution answer questions against the current version of a policy, cite the specific clause that supports the answer, and update behaviour by updating documents rather than retraining a model. Citations matter especially: an answer a customer-facing or compliance user can trace back to a named source is defensible in a way that an ungrounded assertion never is.

Mistake — hard-split chunking breaks regulatory clauses

A naive 500-token hard split cuts a regulatory clause in half. The retrieved chunk reads "the bank may not…" while the rest — "…without prior written consent from APRA" — sits in the next chunk and is never retrieved. The model then generates the exact opposite of the actual rule, with full confidence and a citation that appears to support it. This is among the most dangerous failure modes in regulated RAG precisely because it looks correct.

Fix

Use semantic chunking that splits on section and paragraph boundaries, with a sliding-window overlap of roughly 100 tokens so clause boundaries are never severed. Store document-hierarchy metadata so the parent section can be retrieved alongside any chunk, and run a faithfulness check that flags answers not fully supported by retrieved text. Grounding without faithfulness validation is grounding you cannot rely on.

BFSI use cases: *policy Q&A (APRA / MAS / RBA) · product knowledge base · contract review · regulatory change alerts*

[Reflect] Self-Reflection

Self-Reflection has the agent evaluate its own output against defined criteria — accuracy, completeness, schema conformance, policy compliance — and iterate before returning anything to the caller. It is a quality gate inside a single agent, reducing the hallucinations and format errors that single-pass generation routinely produces. Where Chain of Thought improves the first attempt, Self-Reflection catches the cases where the first attempt was wrong anyway.

How it works

The agent generates a draft, then critiques it against an explicit rubric: is every claim grounded in retrieved context, does the output conform to the required schema, are the regulatory references correct? If any criterion fails, it regenerates the flagged portions. A hard cap — typically three rounds — prevents the loop from running indefinitely, and on cap breach the item is routed to a human rather than silently returned.

```
// Self-reflection loop
1. Generate: agent produces an initial draft
2. Critique: a separate call evaluates against a rubric:
     - Is every claim grounded in retrieved context?  (Y/N)
     - Does the output comply with the schema?        (Y/N)
     - Are regulatory references cited correctly?     (Y/N)
3. Revise:   if any criterion fails -> regenerate with critique
4. Gate:     max 3 rounds -> human review on cap breach
```

Why it matters in BFSI

For regulatory reports, customer communications, and generated documents, a single-pass draft carries quality risk that compounds at scale. Self-Reflection lets the institution embed its review standards directly into the generation process — the same checks a human reviewer would apply, applied first by the system. The result is fewer errors reaching the human, and a clearer record that the checks were performed.

Mistake — critiquing in the same context window

Using the same model instance, in the same conversation, to both generate and critique creates confirmation bias: the model tends to agree with its own output rather than challenge it. This is especially dangerous in compliance review, where independent challenge is the entire point of the exercise. A reviewer who is also the author is not a reviewer.

Fix

Run the critique as a separate call with a different system prompt that frames the model as an adversarial reviewer *("You are a strict compliance reviewer. Your job is to find flaws…")*. In high-stakes flows such as credit and AML, use a different model entirely for the critique step, so that the failure modes of the generator are not shared by the reviewer.

BFSI use cases: *regulatory reports · customer communications · document generation · compliance review*

[Memory] Memory Management

Memory Management governs the agent's state across turns and sessions. Most useful banking interactions are not one-shot; they unfold over a conversation, a case, or a relationship that spans months. The pattern organises state into three tiers: in-context (short-term), an external store (long-term), and episodic memory (compressed history of past sessions). Done well, it gives the agent continuity. Done carelessly, it becomes a latent data-protection liability.

How it works

Tier 1 is the current conversation window — fast but bounded by the context limit. Tier 2 is durable storage, a cache or vector store or relational store keyed by session or customer identifier. Tier 3 holds compressed summaries of prior sessions, distilling long histories into a few salient facts. At session start, relevant Tier 2 and Tier 3 content is loaded into context so the agent resumes with appropriate awareness rather than from zero.

```
Tier 1 — In-context: current conversation window (tokens)
Tier 2 — External:   Redis / vector DB / relational store,
                     keyed by session_id or customer_id
Tier 3 — Episodic:   compressed summaries of past sessions,
                     e.g. "Customer disputed 3 transactions in
                     2024, all resolved in their favour"

Injection: at session start, load Tier 2 + Tier 3 into context.
```

Why it matters in BFSI

An advisor agent that remembers a customer's prior disputes, a hardship flag, or a stated preference delivers materially better service and avoids re-traumatising customers by asking them to repeat themselves. Long-running cases — complaints, AML investigations, collections — depend on continuity across handoffs. But every piece of remembered state is also data the institution is now responsible for under the Privacy Act and relevant data-sovereignty obligations.

Mistake — storing raw PII in AI memory

The agent stores full customer personal information in a vector store that may be hosted in a region violating APRA data-sovereignty expectations or MAS outsourcing-notice obligations. A memory design without data classification is a compliance incident waiting for an audit to discover it. The convenience of "just store everything" is borrowed against a future breach notification.

Fix

Classify every item at write time as PII, sensitive, or public. Store only tokenised references in the AI memory store; the actual personal data stays in your governed, correctly located systems of record, and the memory holds nothing more than the foreign key needed to retrieve it under proper controls. The agent gains continuity; the institution retains custody.

BFSI use cases: *customer context · long-running cases · advisor continuity · dispute tracking*

[Prompt] Structured Prompting

Structured Prompting is the engineering discipline of writing prompts that are deterministic, testable, and maintainable. It is the least glamorous pattern in this paper and arguably the most important, because every other pattern depends on it. A prompt is not a sentence you type once; in a regulated system it is configuration that determines behaviour, and it deserves the same rigour as any other production configuration.

How it works

Structured Prompting combines role framing, enforced output schemas, few-shot examples, and a clear instruction hierarchy. That hierarchy has three levels of authority. Operator instructions — the system prompt encoding the institution's rules and constraints — sit highest. User instructions — the incoming query — sit below them. Retrieved content — RAG context injected at runtime — sits lowest. Higher levels always override lower ones, which is what prevents a cleverly worded user message or a poisoned document from overriding the bank's own rules.

```
// Output-schema enforcement — never accept free-form text
// from a regulated decision agent:
"You must respond ONLY with valid JSON matching:
 {
   decision:        APPROVE | DECLINE | REFER,
   confidence:      number (0.0 - 1.0),
   reasoning_steps: string[],
   regulatory_refs: string[],
   flags:           string[]
 }
 Never add prose outside the JSON object."
```

Why it matters in BFSI

Enforced output schemas turn an unpredictable text generator into a component that downstream systems can consume safely. The instruction hierarchy is a security control as much as a quality one: it is the structural defence against prompt injection, because it establishes that the operator's rules cannot be displaced by anything arriving in user input or retrieved text. And a versioned prompt is what lets an institution answer the question "what instruction was the model operating under when it made this decision?"

Mistake — unversioned prompts in application code

The system prompt lives as a string constant in the codebase, edited by developers without review. Three months later the credit agent begins declining more applications and nobody can say why — the prompt changed subtly in a pull request that no one connected to the outcome. This is a model-governance failure indistinguishable, from the regulator's seat, from an uncontrolled change to a lending policy.

Fix

Treat system prompts as versioned configuration artefacts. Store them in configuration management (AWS SSM Parameter Store, Azure App Configuration, or equivalent), require change approval for regulated use cases, and log which prompt version was active for every inference. The prompt is policy expressed in natural language; govern its change the way you govern any policy change.

BFSI use cases: *every agent — the foundation for all patterns · regulatory compliance · consistent outputs*

Layer 2 — Orchestration Patterns

The coordination layer — how agents work together.

A single well-designed agent is a reliable worker. A network of poorly coordinated agents is an organisational disaster dressed up as innovation. The Orchestration layer is where architects define the structure of multi-agent work: who plans, who executes, who reviews, and who decides that enough has been done. These patterns are the difference between a collection of agents and a system with genuine, governable capability.

The instinct to build one large agent that does everything is understandable and almost always wrong. A monolithic "do everything" agent is brittle, hard to test, hard to reason about, and impossible to govern — because you cannot point to the component responsible for a given decision when responsibility is smeared across one undifferentiated prompt. Orchestration is the discipline of decomposition: many narrow agents, each with one job, a clear input/output contract, and a bounded set of tools.

> **Enterprise principle**
>
> **Specialise agents — do not generalise them.** Narrow agents with one job, clear contracts, and bounded tools are testable, replaceable, and auditable. Those properties matter enormously in regulated environments, where you must be able to say exactly which component did what, and change one without destabilising the rest.

[Plan] Planner–Executor

In the Planner–Executor pattern, a Planner agent decomposes a high-level goal into a structured task graph; Executor agents carry out individual tasks and report back; and the Planner synthesises their results into a final response. It is the natural pattern for any banking process that is genuinely a process — a sequence of dependent steps with a defined outcome — rather than a single question.

How it works

The Planner produces a directed acyclic graph of tasks with explicit dependencies. Tasks with no dependencies run in parallel; dependent tasks wait for their predecessors. This maps directly onto the capability decomposition that enterprise architects already practise in frameworks such as TOGAF: the Planner is doing, at runtime, the same structured breakdown an architect does at design time.

```
// Goal: "Onboard new SME customer and set up trade finance"
// Planner output — task graph (DAG):
T1: KYC_agent    -> verify identity documents
T2: AML_agent    -> run transaction-monitoring profile
T3: CREDIT_agent -> assess facility limit      (depends T1, T2)
T4: DOCS_agent   -> generate facility agreement (depends T3)
T5: NOTIFY_agent -> send welcome package        (depends T4)

// T1 and T2 run in parallel.  T3-T5 run in sequence.
```

Why it matters in BFSI

Customer onboarding, loan origination, trade-finance setup, and regulatory submissions are all multi-step processes with hard ordering constraints. Modelling them as an explicit graph rather than implicit application logic makes the ordering visible, enforceable, and auditable. The graph is documentation, control, and audit trail in one artefact.

Mistake — no DAG enforcement; tasks execute in the wrong order

Credit assessment runs before identity verification completes. In banking, a credit decision made without completed KYC is a regulatory breach under AML/CTF obligations, not merely a bug. This happens whenever the task plan is modelled as a list to iterate rather than a graph with enforced dependencies — the ordering becomes a matter of luck and timing.

Fix

Model the plan as a true DAG and enforce dependency ordering with a workflow engine (Temporal, AWS Step Functions, or equivalent) rather than ad-hoc asynchronous logic in application code. The engine guarantees the ordering and, as a valuable side effect, becomes your durable audit trail of which task ran when, with what inputs, in what order — exactly the record a regulator will request.

BFSI use cases: *customer onboarding · loan origination · trade-finance setup · regulatory submissions*

[Super] Supervisor–Worker

The Supervisor–Worker pattern places a Supervisor agent at the front: it receives requests, classifies intent, routes to the appropriate specialist Worker, and aggregates the workers' outputs. Workers are unaware of one another; only the Supervisor coordinates. This is the natural architecture for an enterprise AI platform, where a single entry point must serve many distinct business domains without collapsing them into one over-broad agent.

How it works

The Supervisor performs two roles — intent classifier at the front and response aggregator at the back. Each Worker is a narrow specialist with its own bounded tools and prompt. Adding a new domain means adding a Worker, not rewriting the Supervisor; retiring a domain means removing one. The topology scales by accretion rather than by growth of any single component.

```
Supervisor:  intent classifier + router + response aggregator
   |
   +-- payments_agent     (NPP / BECS / SWIFT queries)
   +-- compliance_agent   (APRA / AML / CTF rule lookup)
   +-- product_agent      (product terms, rates, eligibility)
   +-- analytics_agent    (portfolio data, dashboards)
   +-- escalation_agent   (human handoff orchestration)
```

Why it matters in BFSI

A bank's AI assistant must field questions spanning payments, compliance, products, and analytics, each requiring different knowledge, tools, and guardrails. Supervisor–Worker lets each domain be owned, tested, and governed independently while presenting one coherent interface to the user. It is the platform pattern: it is how an AI capability becomes shared infrastructure rather than a stack of disconnected point solutions.

Mistake — LLM routing without a confidence fallback

On an ambiguous query — "What are the fees?" — the Supervisor confidently routes to the wrong Worker and returns an answer about the wrong product. With no fallback, the system delivers wrong information with full confidence. Under ASIC obligations, incorrect fee information given to a retail customer is a consumer-harm event, regardless of how the error occurred internally.

Fix

Pair LLM-based routing with a confidence threshold. Below a set confidence (for example 0.75), route to a clarification agent that asks the user to disambiguate before any Worker proceeds. Log every routing decision and its confidence for audit, so misrouting can be detected and the classifier improved rather than silently tolerated.

BFSI use cases: *AI banking platform · multi-domain queries · omnichannel assistant · staff tools*

[Critic] Critic–Refiner

Critic–Refiner separates generation from evaluation across two agents. Agent A produces a draft; Agent B evaluates it against quality, accuracy, and compliance criteria; Agent A revises in light of the critique. The pattern produces materially higher-quality output than single-pass generation, and — because the critique is a distinct artefact — it leaves behind a record of what was checked. It is Self-Reflection elevated to the orchestration layer, with genuine separation between author and reviewer.

How it works

The Critic applies a structured rubric: are mandatory fields populated, do totals cross-foot, are regulatory references accurate, are values within expected ranges? The Refiner incorporates the critique and regenerates only the flagged sections. A human reviewer ultimately sees both the final output and the critique log — which means the human is reviewing not just a document but the evidence that the automated checks were performed and passed.

```
// Regulatory report generation
Draft agent:  generates APRA ARS 330.0 statistical return
Critic agent: evaluates —
    - All mandatory line items populated?   Y/N
    - Totals cross-foot correctly?           Y/N
    - Classification consistent?             Y/N
    - Values within expected ranges?         Y/N
Refiner:      incorporates critique, regenerates flagged parts
Gate:         human reviewer sees final output + critique log
```

Why it matters in BFSI

Regulatory returns, customer communications, contract drafts, and risk reports all demand a level of correctness that single-pass generation cannot reliably deliver. Critic–Refiner embeds the institution's review standard into the workflow and front-loads the checking, so the human reviewer's time is spent on judgement rather than on catching mechanical errors the system should have caught.

Mistake — no iteration cap; the refinement loop never ends

The Critic can always find something to improve; the Refiner always revises; the system never terminates. In production this burns compute budget and, worse, delays time-sensitive regulated submissions past their deadlines — a failure mode reliably observed in teams that omit a hard iteration guard.

Fix

Define "done" explicitly (all mandatory checks pass and confidence is at or above a threshold such as 0.90) and impose a hard iteration cap (for example three rounds). On cap breach, route to human review — do not silently return the last revision as though it had passed. "Not yet good enough after three tries" is a meaningful signal that a person should see.

BFSI use cases: *APRA regulatory returns · customer communications · contract drafting · risk reports*

[MapR] Map–Reduce

Map–Reduce splits a large task into independent chunks (Map), processes them in parallel across multiple agent instances, and aggregates the results (Reduce). It is the answer to the context-window problem: when a document or dataset is far too large to fit into a single model call, you divide, process in parallel, and merge. The pattern borrows its name and its logic from distributed data processing, applied here to language work.

How it works

A 400-page ISDA master agreement is split into forty chunks of roughly ten pages each, with deliberate overlap at the boundaries. Each chunk is processed in parallel by a Map agent that extracts the same structured elements — counterparty obligations, termination events, collateral thresholds, governing law. A Reduce agent then merges the forty extractions, deduplicates, and resolves conflicts into a unified summary.

```
// Large contract review — 400-page ISDA agreement
Map:    split into 40 x 10-page chunks (WITH overlap)
        each chunk -> parallel agent call:
        "Extract: counterparty obligations, termination
         events, collateral thresholds, governing law."

Reduce: aggregator merges 40 results, deduplicates,
        resolves conflicts between chunks,
        produces a unified risk summary.
```

Why it matters in BFSI

Large-contract review, portfolio analysis, document classification at scale, and multi-fund reporting all involve volumes of text no single model call can hold. Map–Reduce makes these tractable and fast through parallelism. But the division that makes it possible is also where its characteristic failure lives, at the seams between chunks.

Mistake — no chunk overlap; clause-boundary failures

Two chunks identify different counterparties for the same clause because the clause spans the boundary between them. The Reducer picks one arbitrarily, and the merged output silently drops a contractual obligation — the kind of error that surfaces only in litigation, years later, when the dropped obligation is the one that mattered.

Fix

Use overlapping chunks (a 100–200 token overlap at boundaries) so no clause is severed, *and make the Reduce step explicitly conflict-aware: "if two extractions disagree on the same clause, flag as CONFLICT and present both versions for human review."* Silent conflict resolution is the enemy; surfaced conflict is exactly what you want the human to adjudicate.

BFSI use cases: *large contract review · portfolio analysis · document classification at scale · multi-fund reporting*

Layer 3 — Integration Patterns

The enterprise fabric — connecting AI to real systems.

An AI agent with no connection to real data is a philosophical exercise. The Integration layer is where AI meets your core banking system, your event bus, your API gateway, and your human workforce. It is also where most enterprise AI projects either deliver genuine value or create technical debt that outlasts the original team. The patterns in this layer answer a deceptively simple question: how does intelligence touch the real world without breaking it?

The defining temptation at this layer is to let the AI bypass existing controls in the name of speed — a direct database connection here, a bespoke connector there, a service account with broad permissions because narrowing them is fiddly. Every such shortcut inherits none of the governance the institution has spent years building into its established integration patterns. The discipline of this layer is to resist that temptation.

> **Enterprise principle**
>
> **AI is a consumer of your existing fabric — not a replacement for it.** Your core banking system, event bus, API gateway, and data lake already exist and already carry governance, authentication, rate limiting, and audit. AI agents should integrate through those existing patterns. Bypassing them with direct database access or bespoke connectors inherits none of your existing controls — and reproduces, badly, problems you already solved.

[Tool] Tool / Function Calling

Tool calling is the bridge between language and action. The agent selects and calls registered tools based on the task; the model produces structured arguments; the runtime executes the actual call and returns the result. Everything an agent does in the real world — reading a balance, executing a payment, enriching a record — happens through a tool. The design of the tool registry is therefore one of the highest-leverage decisions in the entire architecture.

How it works

Each tool in the registry carries a name, a description, a parameter schema, an authorisation scope, a data classification, and a rate limit. The model reads these definitions and decides which tool to call and with what arguments. The quality of the descriptions is the primary lever for reliability — an agent calls the wrong tool far more often because the description was ambiguous than because the model was incapable.

```
// Each tool definition must include:
{
  name: 'get_account_balance',
  description: 'Returns current balance and available funds
                for a given account. Requires auth token.',
  parameters: {
    account_id: { type: 'string', required: true },
    currency:   { type: 'string', enum: ['AUD','USD','SGD'] }
  },
  auth_scope:  'accounts:read',
  data_class:  'PII_FINANCIAL',
  rate_limit:  '100/min'
}
```

Why it matters in BFSI

Tool calling is how AI integrates with core banking APIs, payment rails, customer records, and external data feeds — through the same governed interfaces the rest of the enterprise uses. Treating each tool as a first-class governed object, with explicit scope and classification, is what lets the institution apply least privilege, rate limiting, and audit to AI actions exactly as it does to any other API consumer.

Mistake — the agent has write access it does not need

The tool registry includes update_customer_record alongside the read-only tools, “just in case.” A prompt injection planted in a customer-provided field then induces the agent to call the write tool. The result is a data-integrity event in the core banking system — and a CPS 234 information-security incident — caused not by a model flaw but by an over-broad tool grant.

Fix

Apply least privilege to every agent’s tool set. Read-only agents receive read-only tools. Tools that mutate state require explicit human confirmation (see Human-in-the-Loop). Audit every tool call with the agent’s full session context, so any action can be traced to the conversation that produced it. The blast radius of a compromised agent is exactly the set of tools you gave it — so give it less.

BFSI use cases: *core banking API integration · payment execution · customer record enrichment · external data feeds*

[Event] Event-Driven AI

Event-Driven AI activates agents in response to business events rather than user requests. A payment is initiated, a threshold is breached, a position moves — and an agent is invoked to assess it in real time. The pattern embeds AI inside the institution's existing event architecture rather than forcing a redesign around AI, which is what makes it deployable in environments where the event backbone already carries the bank's most critical flows.

How it works

A consumer on an event stream — a Kafka topic, for instance — triggers an agent when an event matches defined risk signals. For real-time payment fraud, the agent enriches the event with device history, payee velocity, behavioural signals, and geographic anomaly data, then returns a risk verdict. The entire loop must complete inside the payment window the rails impose.

```
// Real-time payment fraud — event-driven activation
Event:   topic 'payments.npp.outbound'
         { amount: 45000, beneficiary: 'new payee',
           time: '02:14', device: 'new mobile' }
Trigger: rule engine flags high-risk signals >= 3
Agent:   fraud_analysis_agent(event_payload)
Enrich:  device history + payee velocity + geo anomaly
Output:  { risk: 0.91, action: 'HOLD_SOFT',
           reason: '3 high-risk signals + new payee' }
Timeline: < 800ms end-to-end, within the NPP 5-second window
```

Why it matters in BFSI

Real-time fraud scoring on the New Payments Platform, AML transaction monitoring, threshold-breach alerting, and liquidity-event response are all event-shaped problems. Event-Driven AI lets the institution apply intelligence at the moment a decision is needed, on the infrastructure it already operates. But "the moment a decision is needed" comes with a hard deadline, and that deadline is where the pattern most often fails.

Mistake — agent latency exceeds the payment SLA window

NPP requires a response within five seconds. LLM inference plus tool calls plus reranking takes six to nine. The agent times out, and the payment either passes unchecked or fails the customer outright. In an NPP Connected Institution context this is a service-reliability incident, and putting an unbounded LLM call on the synchronous payment path is the root cause.

Fix

Design a tiered response. A sub-200ms rule-based pre-screen handles the obvious cases; sub-1s lightweight ML scoring handles the common ones; and asynchronous deep AI analysis is reserved for borderline cases only, taken off the synchronous path. Do not place LLM inference on the synchronous payment path unless its latency is contractually guaranteed. The right architecture is fast-by-default, deep-when-warranted.

BFSI use cases: *NPP real-time fraud scoring · AML transaction monitoring · threshold-breach alerts · liquidity events*

[Cache] Semantic Cache

A Semantic Cache stores AI responses keyed by meaning rather than exact string match. Two questions that differ in wording but ask the same thing hit the same cache entry. For high-volume, repetitive banking queries — product information, rate lookups, policy explanations — this dramatically reduces both latency and inference cost, because a large fraction of traffic is semantically equivalent to something already answered.

How it works

The incoming query is embedded into a vector and compared against cached entries by cosine similarity. A hit above the threshold (for example 0.94) returns the cached response in milliseconds. A miss runs the full pipeline and populates the cache with a configured time-to-live. The threshold is the critical tuning parameter: too low and the cache returns answers to questions that were not really asked; too high and the hit rate collapses.

```
Incoming: 'What is the current home loan interest rate?'
Step 1: embed query -> vector (1536-dim)
Step 2: search cache by cosine similarity
        HIT  (similarity >= 0.94) -> return in ~5ms
        MISS (similarity <  0.94) -> run full pipeline
Step 3: store result in cache with TTL = 3600s

// 60-80% of banking FAQ traffic is semantically
// equivalent to a prior query — hence the large saving.
```

Why it matters in BFSI

FAQ deflection, product-information queries, rate lookups, and policy explanations make up a substantial share of contact-centre and assistant volume, and most of it is repetitive. A semantic cache turns that repetition into a cost and latency advantage. But the very thing that makes it efficient — returning a stored answer to a similar question — is catastrophic if the answer was specific to a person.

Mistake — caching personalised or account-specific responses

"What is MY account balance?" gets cached. The next customer with a semantically similar query receives someone else's balance. This is not a minor bug; it is a catastrophic privacy breach, reportable to the OAIC under the Privacy Act 1988, arising from a caching layer that treated a personalised answer as if it were general information.

Fix

Classify queries at cache-key time. Queries containing personalisation signals ("my", "account", "balance", "transaction") or real-time signals ("current", "now", "today") are cache-ineligible by rule. Only generic, non-personal informational queries ever enter the semantic cache. The default must be ineligible, with eligibility earned, not the reverse.

BFSI use cases: *FAQ deflection · product-information queries · rate lookups · policy explanations*

[HITL] Human-in-the-Loop

Human-in-the-Loop defines explicit checkpoints where human judgement is required before the agent proceeds. It is essential for high-stakes decisions, edge cases, and the many regulatory obligations that mandate human accountability for outcomes affecting customers. The pattern is simple to state and frequently mis-implemented, because the difference between a real HITL gate and a cosmetic one is structural, not visual.

How it works

HITL is implemented as a blocking gate in the workflow graph, not as a notification. The agent produces a recommendation; the workflow pauses; a human approves or rejects; and only then does the agent execute. The workflow engine enforces the pause — the action is structurally incapable of occurring before the human decision, rather than merely discouraged by an application flag the code might bypass.

```
// HITL trigger criteria for BFSI — always escalate if:
  - Agent confidence < 0.75 on a financial decision
  - Transaction value > approval threshold (e.g. $500k)
  - Customer financial-hardship flag = true
  - Potential PEP, sanctions, or adverse-media match
  - Agent attempted the same action >= 3 times unsuccessfully
  - Regulated disclosure required (IDR / EDR obligation)
  - Customer explicitly requests a human
```

Why it matters in BFSI

High-value decisions, regulated disclosures, financial-hardship cases, and AML escalations carry obligations that the institution cannot delegate to a model. HITL is how human accountability is preserved at exactly the points where it is required. It is also, properly placed, the safety net that would have caught the erroneous decline in this paper's opening story before the customer ever saw it.

Mistake — HITL as an afterthought, not a design primitive

The team builds the full pipeline and adds a "flag for review" button at the end. But by the time the flag fires, the agent has already acted — sent the email, updated the record, submitted the payment. The review reviews a fait accompli. Under ASIC's RG 271 internal-dispute-resolution requirements, the customer has already been harmed, and the "human in the loop" was standing beside the loop, not inside it.

Fix

Design HITL as a blocking gate in the workflow graph from the first architecture review. Agent recommends; workflow pauses; human decides; agent executes — in that order, enforced by a workflow engine (Temporal, Step Functions), never by a boolean flag in a service. If the human approval can be skipped by a code path, it is not a gate.

BFSI use cases: *high-value decisions · regulated disclosures · financial-hardship cases · AML escalations*

Layer 4 — Governance & Resilience Patterns

The operating layer — safe, auditable, and reliable at scale.

In the early hours of a Monday morning, when a critical AI service degrades and the fallback routes ten thousand customer queries to a generic error message, it becomes clear that governance was not optional — it was the entire architecture. The Governance layer is the operational wrapper that transforms an AI prototype into a production system that your chief information security officer, your risk committee, and your prudential regulator can stand behind.

This layer is where AI stops being treated as a special, exotic technology and starts being treated as what it is: a critical IT system subject to the same operational-risk, security, change-management, and resilience disciplines as any other. The patterns here are not AI-specific inventions; they are the AI-specific expression of controls the institution already applies elsewhere. The novelty is in the signals — prompt versions, token budgets, groundedness — not in the principle that critical systems must be governed.

> **Enterprise principle**
>
> **Govern the AI system as you govern any critical IT system.** APRA CPS 230 (operational risk) and CPS 234 (information security) apply to AI systems just as they apply to core banking. Your framework must address model change management, data lineage, incident response, vendor-concentration risk from reliance on a single model provider, and third-party assurance over those providers.

[Guard] Guardrails Framework

Guardrails intercept and validate both the inputs to and the outputs from agents. They block policy violations, PII leakage, hallucinated facts, prompt-injection attempts, and out-of-scope responses before they reach end users or downstream systems. Guardrails are a security control, not a user-experience feature — a distinction that matters because it determines where they are applied and how seriously they are maintained.

How it works

Two sets of controls operate at the boundaries of every agent. Input guardrails screen what goes in: prompt-injection detection, PII redaction before any external model call, topic-scope checks, and token-budget enforcement. Output guardrails screen what comes out: groundedness checks against retrieved context, PII-leakage scanning, regulatory-accuracy checks on claims such as rates, and format validation against the required schema.

```
// INPUT guardrails:
  - Prompt-injection detection (regex + LLM classifier)
  - PII detection -> redact before sending to external LLM
  - Topic-scope check -> block out-of-scope queries
  - Token-budget enforcement -> reject > 4000-token inputs

// OUTPUT guardrails:
  - Hallucination detection (groundedness vs retrieved context)
  - PII-leakage scan -> mask before returning to client
  - Regulatory-accuracy check -> flag unverified rate claims
  - Format validation -> enforce JSON-schema compliance
```

Why it matters in BFSI

Every regulated decision flow, every customer-facing channel, and every internal staff tool needs a consistent safety baseline. Guardrails provide it uniformly, independent of the agent's purpose. They are the layer that stops a poisoned input from becoming a harmful action, and a model's plausible-but-wrong output from reaching a customer as fact.

Mistake — guardrails only on customer-facing agents

Internal staff tools are left ungoverned on the assumption that "staff won't misuse them." A prompt injection in an analyst's query then extracts full customer records through the internal KYC agent. Insider threat combined with ungoverned tooling is an OAIC-notifiable data breach, and the assumption that the internal threat model is benign is precisely the gap the attacker uses.

Fix

Apply guardrails uniformly across all agents, internal and external. The threat model for internal tools includes compromised credentials, insider threat, and accidental misuse — none of which respect the customer-facing boundary. Guardrails are a security control applied at every boundary, not a politeness layer for the public-facing ones.

BFSI use cases: *all agents — mandatory baseline · regulated decision flows · customer-facing channels · internal staff tools*

[CB] Circuit Breaker

The Circuit Breaker monitors AI service health and, when error rate or latency crosses a threshold, opens — routing requests to a meaningful fallback rather than failing the customer. Borrowed from electrical engineering and long established in distributed systems, it acknowledges a hard truth: the AI service will sometimes degrade, and the architecture's job is to ensure that degradation does not become customer harm.

How it works

The breaker has three states. Closed is normal operation: requests route to the AI. Open is the degraded state: requests route to a fallback. Half-open is the recovery probe: a small fraction of traffic is sent to the AI to test whether it has recovered, and the breaker returns to closed only if errors have normalised. The decisive design choice is the fallback itself — it must solve the customer's problem, not merely report the technical failure.

```
CLOSED (normal):  error_rate < 5% in last 60s -> route to AI
OPEN (degraded):  error_rate >= 5% -> route to fallback:
    Fallback A: deterministic rules engine (zero latency)
    Fallback B: cached last-known-good response
    Fallback C: HITL queue (human answers within SLA)
HALF-OPEN (probe): after 30s, send 10% of traffic to AI;
    if error normalises -> return to CLOSED
```

Why it matters in BFSI

Critical customer channels, payment services, fraud scoring, and regulatory-reporting pipelines cannot simply stop when the AI component falters. The Circuit Breaker keeps the service available by degrading gracefully to an alternative that still serves the customer — a rules engine, a cached answer, or a human queue with a published SLA.

Mistake — the circuit opens to a generic error, not a real fallback

The breaker opens and routes to "service unavailable." In retail banking, customers stuck in an error loop escalate to complaints. A circuit breaker without a meaningful fallback is worse than none: it fails more predictably, which is not the same as failing gracefully. Predictable failure is still failure from the customer's seat.

Fix

Design fallbacks that preserve the customer experience. For a product-query agent, fall back to FAQ search. For a dispute agent, fall back to a structured intake form feeding a human review queue with a published SLA. The test of a fallback is simple: does it solve the customer's problem? If the answer is no, it is an error page wearing a circuit breaker's clothes.

BFSI use cases: *critical customer channels · payment services · fraud scoring · regulatory-reporting pipelines*

[Obs] AI Observability

AI Observability instruments every agent decision for debugging, performance monitoring, and — above all in a regulated institution — audit. It goes beyond conventional application performance monitoring to capture model-specific signals: token usage, prompt versions, tool-call chains, confidence, guardrail hits, and decision drift over time. It is the pattern whose absence defined the failure in this paper's opening story.

How it works

Every inference produces a structured trace and writes it to an immutable, append-only audit log whose retention matches the institution's record-keeping obligations. The trace captures enough to reconstruct the decision in full: which agent, which prompt version, which model, the token counts, the latency, the tools called and their responses, the decision and confidence, any guardrail hits, and a tokenised reference to the user — never raw PII in the log itself.

```
// Minimum observable data per inference:
{
  trace_id:       'uuid',
  agent_id:       'kyc_enrichment_v2.1',
  prompt_version: 'kyc-sys-prompt-v14',
  model:          'claude-sonnet-4-6',
  input_tokens:   1842,  output_tokens: 312,
  latency_ms:     1240,
  tools_called:   ['sanctions_check', 'pep_lookup'],
  decision:       'HIGH_RISK',  confidence: 0.91,
  guardrail_hits: [],
  user_id:        '[tokenised]',
  timestamp:      '2026-06-05T03:14:00Z'
}
```

Why it matters in BFSI

When an APRA reviewer asks why the system declined a specific customer, the institution must be able to reconstruct that one decision in full. That requires not just the output but the entire context that produced it. Observability is also how model drift is detected before it becomes a pattern of unfair outcomes, and how incidents are investigated rather than merely apologised for.

Mistake — logging the output but not the prompt context

You can see what the agent decided but not why. When the reviewer asks for the reasoning, you can produce only the verdict. Without the prompt version and the retrieved context, the decision cannot be reconstructed — a CPS 230 model-explainability gap that turns a single bad decision into a finding about the whole control environment.

Fix

Log the full inference context — system-prompt version, user input, retrieved chunks, complete model output, and every tool call and response — to an immutable audit log. Set retention to match your record-keeping obligations: in Australia, seven years for regulated decisions. The trace is not telemetry you keep if convenient; it is the evidence you are obliged to produce.

BFSI use cases: *regulatory audit trail · performance dashboards · model-drift detection · incident investigation*

[Cost] Cost & Token Governance

Cost and Token Governance enforces per-agent, per-use-case, and per-team inference budgets, routes each request to the cheapest model tier that meets its quality requirement, and prevents runaway costs from agentic loops or oversized context windows. It is the pattern that keeps an AI programme economically alive long enough to prove its value — and the one most often neglected until Finance notices the bill.

How it works

Each task is classified by complexity at design time and assigned to a model tier: simple lookups to a small model, document analysis and drafting to a mid-tier model, complex regulated reasoning to a top-tier model. Budgets are set per session, per agent, and per team, with an alert at 80 percent consumption and a hard stop at 100 percent that routes to a deterministic fallback. Cost is attributed weekly to business units so that ownership is clear.

```
// Task classification -> model-tier routing:
  Simple lookup / FAQ        -> small model  (Haiku-class)
  Document analysis / draft  -> mid-tier     (Sonnet-class)
  Complex regulated reasoning-> top-tier     (Opus-class)

// Cost guardrails:
  Per-session token budget: 50,000 tokens
  Per-agent daily budget:   $500 AUD
  Alert threshold:          80% consumed
  Hard stop:                100% -> deterministic fallback
  Reporting:                weekly cost by business unit
```

Why it matters in BFSI

At banking scale — millions of interactions a day — model-tier choice is the difference between a sustainable programme and one Finance shuts down before it proves itself. Cost governance also intersects with operational risk: an unbounded agentic loop is both a runaway cost and a CPS 230 concern, and the same controls that cap the bill cap the blast radius.

Mistake — using the most capable model for every task

Every request routes to the most capable, most expensive model because “quality matters.” A large bank running two million interactions a day at top-tier pricing generates a multimillion-dollar monthly inference bill before any business value is proven, and the programme is cancelled on cost grounds before it had a chance to demonstrate worth.

Fix

Map task complexity to model tier at design time, and benchmark the quality gap for your specific use cases. For FAQ and triage, a small model at a fraction of the cost typically achieves well over 90 percent of the quality. Reserve the largest models for the complex, regulated decisions that genuinely need them. Quality matters — which is exactly why you should spend your quality budget where it changes outcomes.

BFSI use cases: *FinOps reporting · model-tier routing · budget enforcement · vendor-concentration management*

Mapping Patterns to Regulatory Obligations

The patterns in this paper are jurisdiction-neutral engineering practices. The obligations they help an institution discharge are not. This chapter draws the line between the two explicitly, so that an architect can answer the question a risk officer will inevitably ask: which control does this pattern actually satisfy?

The mapping is deliberately conservative. A pattern "helping discharge" an obligation does not mean it satisfies that obligation by itself; it means the pattern is a necessary part of a control that, combined with policy, process, and human oversight, does. Patterns are the implementation layer of governance, not a substitute for it.

Obligation	What it requires of AI systems	Patterns that help discharge it
APRA CPS 230 (Operational Risk)	Identify, manage and control operational risk in critical operations; demonstrate processes behave as intended; maintain records.	Observability · Chain of Thought · Circuit Breaker · Planner–Executor
APRA CPS 234 (Information Security)	Protect information assets; maintain controls proportionate to threat; manage third-party security.	Guardrails · Tool Calling (least privilege) · Memory (data classification)
ASIC RG 271 (Internal Dispute Resolution)	Handle complaints fairly and promptly; preserve human accountability for customer outcomes.	Human-in-the-Loop · Circuit Breaker · Supervisor–Worker (confidence gate)
MAS TRM Guidelines	Sound technology-risk governance; resilience; auditability; managed outsourcing.	Observability · Circuit Breaker · Chain of Thought · Memory (sovereignty)
Privacy Act 1988 (Australia)	Protect personal information; notify eligible data breaches to the OAIC.	Semantic Cache (classification) · Memory (tokenisation) · Guardrails (PII)
ISO/IEC 42001 (AI Management)	Establish an AI management system: governance, lifecycle, risk, transparency.	All four layers — the patterns are the control implementation
EU AI Act (high-risk systems)	Risk management, logging, human oversight, accuracy and robustness for high-risk AI.	Observability · Human-in-the-Loop · Guardrails · Self-Reflection

A note on vendor-concentration risk. Reliance on a single model provider is itself an operational-risk concentration under CPS 230 and a third-party-management concern under both CPS 234 and the MAS TRM guidelines. The Cost & Token Governance and Circuit Breaker patterns, taken together, are also the architectural levers for managing it: tier routing that can target more than one provider, and fallbacks that do not assume any single model is available. Build the abstraction that lets you change providers before you are forced to.

An Adoption Roadmap

Patterns are learned by applying them in the right order. The following phased roadmap reflects how mature institutions sequence adoption — foundations and governance first, capability and scale second. It is deliberately weighted toward establishing trust before extending reach, because an AI programme that loses the confidence of its risk function does not get a second chance.

Phase 1 — Establish the trustworthy primitive (months 0–3)

Stand up a single, narrow, genuinely useful agent and do it properly. The goal is not breadth but a reference implementation of the foundations that every later agent will inherit.

- Implement Structured Prompting with versioned prompts in configuration management from day one.
- Add Chain of Thought and full AI Observability so every decision is explainable and reconstructable.
- Ground the agent with RAG using semantic chunking; validate faithfulness.
- Wrap it in a baseline Guardrails framework covering input and output.
- **Exit criterion:** you can reconstruct any single decision the agent made last week, in full, from the audit log alone.

Phase 2 — Make it safe to operate (months 3–6)

Before adding capability, add the controls that let the institution operate the agent without holding its breath.

- Introduce Human-in-the-Loop as a blocking gate on high-stakes actions.
- Add a Circuit Breaker with a fallback that genuinely serves the customer.
- Apply least-privilege Tool Calling; remove every permission the agent does not strictly need.
- Stand up Cost & Token Governance with budgets, alerts, and a hard stop.
- **Exit criterion:** a model-provider outage degrades the experience but harms no customer, and a runaway loop hits a budget stop rather than the news.

Phase 3 — Compose and scale (months 6–12)

With trustworthy, governed primitives in place, orchestration and scale become safe to pursue.

- Introduce Supervisor–Worker to serve multiple domains behind one interface, with confidence-gated routing.
- Use Planner–Executor on a true workflow engine for multi-step processes such as onboarding.
- Add Critic–Refiner with independent review for generated regulatory and customer content.
- Apply Map–Reduce to large-document workloads; Event-Driven AI to real-time flows with tiered latency.
- Add Semantic Cache for high-volume informational traffic, with strict query classification.
- **Exit criterion:** new domains are added by composing existing patterns, and each addition inherits the governance baseline automatically rather than re-implementing it.

> **Sequencing principle**
>
> **Trust before reach.** An institution that scales orchestration on top of ungoverned foundations scales its

risk. One that establishes explainability, safety, and cost control first earns the licence — from its own risk function and its regulator — to extend AI into higher-stakes work.

The Anti-Pattern Index

This is the architect's diagnostic checklist — the one from this paper's opening story, generalised. Read it as symptom-to-cause. When a system misbehaves in production, the failure is usually not mysterious; it is a known pattern that was skipped. Find the symptom; the missing pattern is beside it.

If you observe this symptom...	...the missing pattern / fix is
Agent burns tokens, loops, never stops	ReAct — add an explicit stopping condition in the runtime, not just the prompt
Cannot explain a decision to a reviewer	Chain of Thought + Observability — persist full reasoning and prompt context
Model confidently states the opposite of a rule	RAG — semantic chunking with overlap; faithfulness validation
Generated reports pass review then fail later	Self-Reflection / Critic–Refiner with an independent reviewer and a cap
Privacy or sovereignty concern in stored state	Memory Management — classify at write time; store tokenised references only
Behaviour changed and nobody knows why	Structured Prompting — version prompts as governed configuration
Tasks run out of order; control after KYC	Planner–Executor — enforce a DAG with a workflow engine
Assistant answers the wrong domain confidently	Supervisor–Worker — confidence threshold + clarification fallback
A large document review drops an obligation	Map–Reduce — overlapping chunks + conflict-aware reduce step
Agent took a write action it should not have	Tool Calling — least privilege; human confirmation for mutations
Real-time scoring misses the payment SLA	Event-Driven AI — tiered response; keep LLMs off the synchronous path
One customer sees another's data from cache	Semantic Cache — classify queries; personalised queries are cache-ineligible
Harm occurs before the review fires	Human-in-the-Loop — make it a blocking gate, not a notification
Internal tool leaks records via a crafted query	Guardrails — apply uniformly to internal and external agents
Outage routes customers to a dead error page	Circuit Breaker — design a fallback that solves the problem
Inference bill threatens the programme	Cost & Token Governance — tier by complexity; budget and hard-stop

Glossary

Agent — A bounded AI component that reasons over inputs and acts through tools to accomplish a defined task, with its own prompt, contract, and permissions.

Agentic loop — The repeating cycle of reason, act, observe by which an agent makes progress — the core of the ReAct pattern.

Chunking — Dividing a document into pieces for embedding and retrieval. Semantic chunking splits on meaning-preserving boundaries; hard splitting cuts at fixed token counts and can sever clauses.

Confidence threshold — A minimum confidence below which an agent must escalate, clarify, or defer rather than act — a control against confident error.

DAG — Directed acyclic graph: a task structure with explicit, non-circular dependencies, used to enforce correct execution order in the Planner–Executor pattern.

Embedding — A numeric vector representing the meaning of text, enabling similarity search in retrieval and semantic caching.

Faithfulness — The degree to which a generated answer is supported by the retrieved context rather than the model's memory; the key validation step in RAG.

Groundedness — Whether an output is anchored in provided evidence; checked by output guardrails to detect hallucination.

Guardrail — An input or output control that blocks unsafe, non-compliant, or out-of-scope content at an agent's boundary.

Hallucination — A confident, plausible, but unsupported or false model output — the failure mode RAG and groundedness checks target.

HITL — Human-in-the-loop: a design in which human judgement is structurally required before an agent proceeds, implemented as a blocking workflow gate.

Least privilege — Granting an agent only the tools and permissions strictly required, to minimise the blast radius of misuse or compromise.

Model tier — A capability/cost band of model (small, mid, top); requests are routed to the cheapest tier that meets the quality requirement.

Prompt version — An identified, governed revision of a system prompt, logged per inference so behaviour can be tied to the exact instruction in force.

RAG — Retrieval-Augmented Generation: injecting retrieved external documents into the model's context at query time to ground its answers.

Token — The unit of text a model processes and is billed on; token budgets bound both cost and the length of agentic loops.

Trace — The structured, immutable record of a single inference — prompt version, context, tools, decision, confidence — used for audit and incident investigation.

Vector store — A database that indexes embeddings for similarity search, underpinning RAG retrieval, episodic memory, and semantic caching.

Epilogue: The Architecture Was the Point

The architect who returns from the war room at dawn — the one from our opening story — does not go looking for new technology. She goes looking for the patterns that were missing: the versioned prompt that was not there, the audit log that was stripped, the circuit breaker that routed to nowhere, the human gate that fired too late. Her remediation backlog is, almost line for line, the table of contents of this paper.

The story that opened this guide — a payment declined in error, logs without context, a team searching for an answer that was never stored — is not a cautionary tale about artificial intelligence. It is a cautionary tale about the absence of architecture. The model did exactly what models do. What failed was everything around it: the discipline that should have made its behaviour bounded, explainable, recoverable, and accountable.

The patterns collected here are that architecture. Apply them in the right order: **Foundation before Orchestration; Integration through the existing fabric, not around it; Governance as a design primitive, not a retrofit.** Build the audit trail from day one. Version the prompts. Cap the loops. Gate the writes. Classify the data. Design the fallback before the failure.

None of this is exotic. It is the ordinary discipline of enterprise IT, applied honestly to a technology that is powerful enough to cause real harm when that discipline is absent. The institutions that will deploy AI safely at scale are not the ones with the most advanced models. They are the ones that treated AI, from the first architecture review, as a critical system deserving of the controls every other critical system already has.

The next architect on call at three in the morning will thank you for it.

Enterprise Architecture · Banking & Financial Services · 2026